

Type functions and beyond

An exploration of type functions and concept functions

J. Monnon

2018-02-26

- [I. Introduction](#)
 - [1. Abstract](#)
 - [2. Terminology](#)
 - [3. General considerations](#)
 - [4. Design rules](#)
 - [5. constexpr-based metaprogramming](#)
- [II. Type functions](#)
 - [1. Type traits as functions](#)
 - [2. Overloading](#)
 - [3. Concepts introducing types](#)
 - [4. Overloading \(continued\)](#)
 - [5. Conditionals with if](#)
 - [6. Pattern-matching with switch case](#)
 - [7. Processing sequence of types with for](#)
 - [8. ADL](#)
 - [9. Composition](#)
 - [10. Operator overloading](#)
 - [a. Comparison operators](#)
 - [b. Relational operators](#)
 - [c. Logical operators](#)
 - [d. Composition operator](#)
 - [e. Sum types and product types](#)
 - [11. Metaclasses](#)
 - [a. Metaclasses as type functions](#)
 - [b. .is](#)
 - [c. .as](#)
 - [12. Type attributes and type constructors](#)
 - [a. Type attributes](#)
 - [b. Type constructors](#)
 - [13. Note on the implementation](#)
- [III. Concept functions](#)
 - [1. Optional\(\) and composition](#)
 - [2. Instrumented](#)
- [IV. Further considerations](#)
 - [1. Opportunity for unification](#)
 - [2. On metaprogramming](#)
- [V. Acknowledgements](#)
- [VI. References](#)
 - [\[Elements of Programming\] Alexander Stepanov & Paul McJones, 2009, Addison-Wesley](#)
 - [\[Boost.Hana\] Louis Dionne, *Hana*](#)

- [\[P0425\] Louis Dionne, *Metaprogramming by design, not by accident*](#)
- [\[P0707\] Herb Sutter, *Metaclasses: Generative C++*](#)
- [\[P0343\] Vicente J. Botet Escribá, *Meta-programming High-Order functions*](#)
- [\[cppreference.com\] Constraints and concepts](#)
- [\[P0694\] Bjarne Stroustrup, *Function declarations using concepts*](#)
- [\[P0095\] David Sankel, *Pattern Matching and Language Variants*](#)

I. Introduction

1. Abstract

This document proposes to extend functions to let them operate directly on types and concepts. The goal is to allow writing metaprogramming in the most intuitive and consistent way with the rest of the language.

Here is an example of a type function:

```
ForwardIterator IteratorType(typename T) {
    // In a type function, an `if` behaves as a `if constexpr`.
    if (Container(T)) // `Container` is a concept
        return T::iterator;
    else if (Array(T)) // `Array` is a concept
        return Decay(T);
}

// On call site:
typename I = IteratorType(C);
```

A type function is always executed at compile-time. Here, it takes a type `T` and returns another type that models the `ForwardIterator` concept. Type functions allow a natural and straightforward notation to manipulate types.

They also allow to introduce powerful mechanisms, such as `switch case` to perform pattern-matching:

```
// The codomain is the return type of a mathematical function type.
typename CodomainType(FunctionObject F) { // FunctionObject is a concept, F is a type
    typename Ret; // We declare an unconstrained type
    Class C;      // and a class type (`Class` is a concept)
                  // for the following pattern matching:
    switch (F) {
    case Ret (...): // function
    case Ret (*)(...): // pointer to function
    case Ret (C::*)(...): // pointer to member function
    case Ret (C::*)(...) const: // pointer to const member function
    case Ret (C::*)(...) volatile: // pointer to volatile member function
    // other variants omitted...
        return RemoveCv(RemoveReference(Ret));
    default: // user-defined type
        ...
    }
}
```

This document proposes that the syntax `<concept-name> <name>` always introduces a type, and not an object (e.g. `ForwardIterator I` introduces the type `I`, with `I` having to model the concept `ForwardIterator`). The syntax `<concept-name>{<type-name>} <name>` is proposed to introduce an object whose type must model a given concept (e.g. `ForwardIterator{I} begin` introduces the object `begin` of type `I`, with `I` having to model the concept `ForwardIterator`).

This document also introduces usual function mechanisms for type functions : overloading, ADL and operator overloading.

For example with operator overloading, it is possible to compare two types with `==`:

```

typename MyTypeFunction(FunctionObject F) {
    if (CodomainType(F) == void) {
        // special case...
    } else {
        // ...
    }
}

```

Some more advanced usages of operator overloading includes: | for composition ; &&, ||, ! for predicate types ; +, * for algebraic datatypes.

This is an example of a type function composition:

```

// `RemoveCvReference`, `ValueType`, `CodomainType` are type functions.
// `CodomainOfValueType` is a new type function with `CodomainOfValueType(T)`
// begin equivalent to `CodomainType(ValueType(RemoveCvReference(T)))`.
typename CodomainOfValueType = RemoveCvReference | ValueType | CodomainType;

// C is a container of function objects
template<Container C>
auto do_stuff(C&& functions) {
    std::vector<CodomainOfValueType(C)> results;
    // ...
}

```

A proposition is made to express [\[P0707\]](#)'s metaclasses in terms of type functions, as metaclasses seem to be a subset of type functions (which makes a priori this proposal an alternative and a superset of [\[P0707\]](#)'s metaclasses).

For that, a syntax `->(T) { ... }` is introduced to inject code into a type `T`. The simplest example is for creating strong typedefs (equivalent to metaclass' `.as`):

```

typename new_type(typename T) {
    return ->(T) {}; // injects nothing but creates a new type
}

typename my_T = new_type(T);

```

Type attributes (function associating a type to a compile-time object, such as `sizeof(T)`) are also considered as a variant of type functions, allowing for example to define `nameof(T)`, `membersof(T)`, and so on.

Finally, concept functions are introduced to manipulate and transform concepts. One of the simplest examples of concept function is to create a new concept by adding constraints to an existing one:

```

// Adds the constraints of the `Serialize` concept to any concept.
concept Serializable(concept C) {
    return C && Serialize;
};

// On call site:
template<Serializable(Container) C>
...

// Or (assuming `Instrumented` is another concept function):
template<Instrumented(Serializable(FunctionObject)) F>
...

```

For a greater readability, operators can also be used to compose concept functions:

```

concept InstrumentedSerializable = Serializable | Instrumented;

// `InstrumentedSerializable(FunctionObject)` is now the same concept as
// `Instrumented(Serializable(FunctionObject))`

template<InstrumentedSerializable(FunctionObject) F>
...

```

This document strives to show the consistency of its approach. The features it introduces can be grouped as:

- concepts introducing types only (not objects)
- type functions
- code injection variant ($\rightarrow(T) \{ \dots \}$)
- concept functions

2. Terminology

A note on the terminology used in the rest of this document:

- "concept" means "type-concept": a set of requirements on one or several types.
- "object" means "entity having a type", by opposition to a type itself (for example, in `int i`, `int` is a type and `i` is an object).

3. General considerations

As C++ programmers we typically think of functions as procedures operating on objects. In contrast, a mathematical function is more generally a rule associating to each element of a set (the domain) an element of another set (the codomain). As a rule, a mathematical function doesn't care about the nature of the associated elements. In particular, the level of abstraction of the elements is irrelevant.

In C++, when we write a metaprogram:

- we access *type attributes*, e.g.:
 - we access the size of a type with `sizeof(T)`
 - we access the alignment of a type with `alignof(T)`
 - we access the number of elements of a tuple with `std::tuple_size_v<T>`
- we use *type functions* to associate a type to another type or to transform a type, e.g.:
 - we associate a container type to its iterator type
 - we decay an array type to a pointer type
 - we add members or add/delete methods to a type
- we use *type constructors* to create new types, e.g.:
 - we form a pointer type by appending `*` to an existing type
 - we form a vector type by instantiating the `std::vector` template class

(See [\[Elements of Programming\]](#), section 1.7 for further information about this classification)

All these operations are mathematical functions that associate a type to something else. In the case of type attributes, the type is associated to a (compile-time) value. In the other cases, the type is associated to another type. The fact that these operations are pure mathematical functions is also confirmed by the purely functional nature of template metaprogramming.

Since these operations are functions, it is natural to denote them with the syntax of functions:

```
codomain function(arg0, arg1...)
```

The manipulated elements are not objects anymore, but types. Note that this is already the case with `sizeof(T)` and `alignof(T)`. The important point is not that elements have now a different nature (they are not objects anymore), it is rather that the manipulation pattern is the same (associating elements to other elements). That is, what is proposed here is to make the syntax emphasize the manipulation pattern rather than the nature of the manipulated elements (objects, types...).

This is intuitive and natural: this is what we do all day long by having our perceptions pattern-matching our environment. For example, we can recognize a circle on a road sign or we can express that the same arguments are used again and again in a discussion by saying this discussion goes in circle. We apply the same structural idea

(circle) to elements of different natures (road sign, discussion). This is also a fundamental principle of mathematics: focusing on patterns, behaviors and relationships instead of the internal characteristics of the manipulated elements. This is what makes it general and powerful.

When designing C++, by focusing on patterns rather than the nature of the elements, we simplify the language, make it easier to learn, more consistent and more powerful. Every time we introduce a new entity (object, type, concept...) in the language, one of the questions should be "how can we manipulate it?" and "how does it compose with the other features of the language?". Functions are precisely a good candidate as a manipulation tool because their main characteristic is composability (which is confirmed by a strong theoretical background). Moreover, users already master runtime functions and their features (overloading, ADL...) so the learning cost of extending functions to other entities such as types is minimal.

Of course, it must be noted that it is already possible to somewhat compose operations on types. For example with a cv-qualified integral type N, we can write:

```
std::make_unsigned_t<std::remove_cv_t<N>> i = 0;
```

But these are functions in disguise and would be more naturally written as:

```
MakeUnsigned(RemoveCv(N)) i = 0;
```

This is not only a syntax issue, using type functions allows us to depart from template metaprogramming constraints (instantiation cost, unnatural syntax) and reuse familiar runtime syntax to manipulate types (if, switch case, operators, etc.). This also allows us to reuse runtime function features, such as overloading and ADL (see [section II](#)). From there, it is only natural to consider concept functions, because concepts are also entities to be manipulated by the users (see [section III](#)). Finally, homogenizing the syntax opens the door for future language unifications (e.g. functions able to indifferently operate on object or types, see [section IV](#)).

The goal of this document is to present the design of type functions and concept functions and show how they can change for the better programming in C++.

4. Design rules

This document proposes introducing a new feature in the language, by trying to follow these design rules:

- to be orthogonal with other language features
- to be composable
- to focus on internal consistency
- to be as simple and minimalistic as possible
- to not artificially limit users' possibilities but to offer them a powerful tool they will freely use to solve their problems
- to simplify the language by allowing to use similar constructs for similar patterns, whatever the abstraction layer
- to strive for unification making the language simpler and more powerful
- to not break existing language features

5. constexpr-based metaprogramming

Various approaches try to manipulate types as constexpr values. One such approach is [\[Boost.Hana\]](#) ; another one makes use of "meta" types to inspect types ([\[P0425\]](#)).

In both cases, the idea is to:

1. translate the type to a manipulable form (a constexpr object)
2. manipulate this form
3. translate back to a real type

These approaches have their strengths: they leverage an existing mechanism (constexpr functions) to manipulate types. But they somewhat introduce a syntactic noise obfuscating the user's intent and do not syntactically acknowledge that on a logical level we are applying a function.

For instance, assuming the existence of a constexpr function `partial` that binds some type arguments to a function type, with [\[Boost.Hana\]](#) we write (this example is taken from [\[P0343\]](#)):

```
typename decltype(hana::partial(type_c<Fn>, type_c<Args>...))::type f;
```

With [\[P0425\]](#), the equivalent form is:

```
TYPENAME(decltype(partial(REFLEXPR(Fn), REFLEXPR(Args)...)) f;
```

where `REFLEXPR` yields the constexpr object for a type and `TYPENAME` translates back to a type.

We would prefer to simply write:

```
Partial(Fn, Args...) f;
```

where `Partial` is a type function, operating on types and yielding a type. [Section II](#) shows how users can implement a type function. Some implementations might chose to translate type functions into template code, even if it might seem more natural to translate them into constexpr functions (see [section II. 13](#)).

II. Type functions

This section introduces type functions: function manipulating types. This feature does not conflict with any language feature, except for an adaptation of how concepts introduce names (see [section II. 3](#)). In the following, type functions' features are introduced in an incremental way. Their main benefits may appear to the reader starting from [section II. 5](#).

1. Type traits as functions

Consider a type trait that associates a type to its iterator type. Currently, we typically write this kind of code:

```
// Default case: assume a container type.
template<typename Container>
struct iterator_type {
    using type = typename Container::iterator;
};

// Array specialization.
template<typename T, std::size_t N>
struct iterator_type<T [N]> {
    using type = T*;
};
```

And use it this way with a template type parameter `T`:

```
// A `using` can be used to simplify the writing.
typename iterator_type<T>::type it;
std::vector<typename iterator_type<T>::type> iters;
```

This kind of traits are mathematical functions: they map an element to another one. The corresponding algorithm is:

```
For a type T,
if T is an array of the form U [N]:
    return U*
else
    return T's inner `iterator` typedef
```

This algorithm is simple but currently translates in the language into a verbose and scattered implementation. The more complex the algorithm, the more problematic this situation is.

If we were not at compile-time but at runtime (let's say we write a dynamic type system), we would write something like:

```
// `type` has all info about our dynamic type
type iterator_type(type t) {
    if (is_array(t))
        return decay_type(t);
    else
        return t.inner_type("iterator");
}
```

We would like runtime version and compile-time versions to be as similar as possible (see next section).

2. Overloading

Consider the following type function (implementation considerations can be found in [section II. 13](#)). We first handle the default case:

```
// Default case: assume a container type.
ForwardIterator IteratorType(typename ContainerType) {
    return ContainerType::iterator;
}
```

IteratorType is a function operating at compile-time on types. To specify that it operates on a type and not on an object (i.e. not on an instance of a type), the keyword `typename` is used. When the type itself is constrained, a concept is used.

Here, the type function takes an arbitrary type, i.e. a non-constrained type denoted by `typename`, and returns a type constrained by the `ForwardIterator` concept. Informally, we say that this type function takes any type and returns a forward iterator type.

Since a type function returns a type, it can be used wherever a type is allowed:

```
// C is a container type.
IteratorType(C) it;
std::vector<IteratorType(C)> iters;
// etc.
```

We now overload `IteratorType` for array types:

```
// Array overload
RandomAccessIterator IteratorType(Array T)
{
    return Decay(T);
}
```

`Array` is a concept modeled by any native array type. It introduces a *type* and not an object (see [next section](#) for a discussion on this). `Decay` is another type function whose behavior is equivalent to `std::decay_t`.

It would be useful to be able to do a form of pattern-matching to decompose the array type T into $U[N]$. See [section II. 6](#) for more on this.

We now have two overloads but there is no ambiguity between them since the `Array` concept is more constrained than `typename` (no constraint).

Note that we could use an unconstrained concept in lieu of `typename`:

```

template<typename T>
concept AnyType = true;

// Default case: assume a container type.
ForwardIterator IteratorType(AnyType ContainerType) {
    return ContainerType::iterator;
}

```

Also, this interface is a bit misleading since the default version won't work for any type but only on container types. This is addressed in [section 4](#).

3. Concepts introducing types

The previous example conflicts with C++20 concepts, where AnyType for example would introduce an object and not a type. We think that in the same way as a type introduces an object, a concept should introduce a type:

```

int i;
// `i` is an object.
// The form of the declaration is: type object;

ForwardIterator I;
// `I` is a type.
// The form of the declaration is: concept type;

```

This would allow to keep a sane abstraction order (concept > type > object) by avoiding an "abstraction jump" from concept to object, which is logically unclear. This wouldn't impact other template parameters introduction, because they don't use concepts to introduce objects:

```

// 'template introduction' syntax: ok
ForwardIterator{I}
I algo_1(I begin, I end);

// 'long form' syntax: ok
template<ForwardIterator I, UnaryPredicate P>
I algo_2(I begin, I end, P predicate);

```

Introducing types in a distinct way than objects also has the benefit to be unambiguous to the reader. Consider (taken from [\[cppreference.com\]](#)):

```

void f0(Comparable a, Comparable* b);
// The two `Comparable` introduce the same type.
// long form: template<Comparable T> void f0(T a, T* b);

void f1(auto a, auto* b);
// The two `auto` introduce different types.
// long form: template<typename T, typename U> f1(T a, U* b);

```

and compare them to this form, where information on concepts, types and objects is clearly stated:

```

template<ForwardIterator I0, ForwardIterator I1, UnaryPredicate P>
I1 algo_3(I0 begin0, I0 end0, I1 begin1, P predicate);

```

One solution to have a concise and unambiguous syntax may be to allow the template introduction syntax in-place:

```

ForwardIterator{I1} algo_3(ForwardIterator{I0} begin0, I0 end0,
                          I1 begin1, UnaryPredicate{P} predicate);

// equivalent to the previous form:

template<ForwardIterator I0, ForwardIterator I1, UnaryPredicate P>
I1 algo_3(I0 begin0, I0 end0, I1 begin1, P predicate);

```


Note that `end0` and `begin1` are simply introduced by their respective types `I0` and `I1` without repetition of the concept. This solution is both concise, logically correct and non-ambiguous.

Also note that it is more concise, even compared to Stroustrup's "natural syntax" (see [\[P0694\]](#)):

```
// "natural syntax"
InputIterator find(InputIterator begin, InputIterator end, UnaryPredicate p);

// proposed unambiguous syntax
InputIterator{I} find(I begin, I end, UnaryPredicate{P} p);
```

Optionally, it could be allowed to omit naming the type when it is not needed:

```
// No need to name the container type.
// Here, `container` is an object, not a type.
void algo_4(Container{}& container);
```

The same logic would apply on call site:

```
// Introduces a forward iterator type `I`, and an object `it` of type `I`:
ForwardIterator{I} it = algo_1(begin(container), end(container));
I tmp;
// ...
```

Or as before, if naming the type is not needed:

```
ForwardIterator{} it = algo_1(begin(container), end(container));
```

This is to contrast with the following, where `C` is a container type and `I` is a forward iterator type:

```
// Here, `I` is a type.
ForwardIterator I = IteratorType(C);
```

A clear distinction between objects (introduced only by types) and types (introduced only by concepts) is crucial to be able to manipulate types and define type functions as will be shown in the next sections.

4. Overloading (continued)

Up to this point, we have used overloading to specialize our type functions for families of type (denoted by concepts). What if we want to specialize not for a family of types but for one specific type? We can notice that it can be done by using a concept matching our specific type only.

Going back to `IteratorType`, let's say we create a new type that has an associated iterator type, but that is neither a container type with an internal iterator typedef, nor an array. It could be a range adaptor, not owning its elements, and applying a transformation on the current value before dereferencing.

```
// Assumes `Range` and `Transformation` are concepts.
// A transformation is a function taking a value and
// returning another value of the same type.
template<Range R, Transformation F>
class transformation_range;
```

The iterator type we want to associate to `transformation_range` is:

```
template<Transformation F>
class transformation_iter;
```

First, let's rewrite the default version of `IteratorType` to acknowledge that it effectively only accepts container types:

```
ForwardIterator IteratorType(Container C) {
    return C::iterator;
}
```

Now we specialize `IteratorType` for our new type by assuming a `TransformationRange` concept only modeled by template instantiations of `transformation_range`:

```
template<typename T>
concept TransformationRange =
    ... // constraints
;

// We assume that `TransformationType` is a type function
// that returns the associated transformation type.
ForwardIterator IteratorType(TransformationRange T) {
    return transformation_iter<TransformationType(T)>;
}
```

It is fastidious to write a concept for a unique type (and to write associated type functions). As this case may often happen, we need a way to "lift" a type to the concept level. This means having a way to automatically generate, from a type, a concept modeled by this type only. We propose to do it with a concept built-in function:

```
template<Range R, Transformation F>
ForwardIterator IteratorType(concept(transformation_range<R, F>) T) {
    return transformation_iter<F>;
}

// or with in-place template introduction syntax:
ForwardIterator IteratorType(concept(transformation_range<Range{R}, Transformation{F}>) T) {
    return transformation_iter<F>;
}
```

With a type `X`, `concept(X)` returns a concept so in the example above `T` is a type, which non-ambiguously makes `IteratorType` a type function. This also allows to decompose a type by pattern-matching, in the same way it works with template specialization. In the example above, it allows us to access `F` without a `TransformationType` type function.

We can now call `IteratorType` on `transformation_range` template instantiations:

```
// `Rng` is `transformation_range<R, F>`.
// `R` and `F` are deduced in `IteratorType`.
ForwardIterator I = IteratorType(Rng);
```

5. Conditionals with `if`

Up to this point, we've seen overloading and one-line type functions. If we ignore the function syntax, this is very close to template specialization. Of course, the benefit of functions is shown by more complex examples. Type functions are an opportunity to introduce a more natural syntax for manipulating types.

We previously implemented the `IteratorType` trait with an overloaded type function. But we can also implement the default version as a single type function:

```
ForwardIterator IteratorType(typename T) {
    if (Container(T)) // `Container` is a concept
        return T::iterator;
    else if (Array(T)) // `Array` is a concept
        return Decay(T);
    fail("T is neither a container type nor an array type.");
}
```

We propose that a concept can be used as a type predicate. It returns `true` if the type fulfills all the requirements. Thus

```
Container(T)
```

is equivalent to the current syntax

```
Container<T>()
```

In a type function, all `if` are `constexpr`: a branch is evaluated only if the corresponding condition is true. In the previous example, if `T` is an `Array` the "container" branch is not evaluated, which avoids the ill-formed expression `T::iterator`. This behavior is intuitive and consistent with the behavior of runtime `if`.

We propose that the function `fail()` (naming subject to change) causes a classical "substitution failure", so that the following function

```
template<typename C>
IteratorType(C) f(C& c, IteratorType(C) it);
```

is discarded from the overload set if `IteratorType` fails, possibly displaying the failure message in verbose compilation.

Note: Having a centralized default version doesn't make specialization useless. Users still need to specialize for their types and concepts, as seen with `transformation_range`.

More complex uses of conditionals will be shown in the following sections.

6. Pattern-matching with switch case

Classical template metaprogramming relies on a limited form of pattern-matching: decomposing a type according to the different ways it can be created (pointer, array, function, etc.). For types `T`, `U`, `C` and integral constant `N`, this includes: `T*`, `T&`, `T[N]`, `T(U)`, `T (C::*)(U)`, etc.

Functional programming languages make a heavy use of pattern matching through constructions similar to `switch case`.

We define `switch case` in type functions as a mechanism to perform pattern-matching. Let's rewrite `IteratorType` by leveraging this feature:

```
ForwardIterator IteratorType(typename T) {
    if (Container(T))
        return T::iterator;
    typename U; // declare an unconstrained type,
    std::size_t N; // declare an integral constant
                  // for the following pattern-matching:
    switch (T) {
    case U[N]:
        return U*;
    default:
        fail("T is neither a container type nor an array type.");
    }
}
```

Or by using in-place template introduction syntax:

```
ForwardIterator IteratorType(typename T) {
    if (Container(T))
        return T::iterator;
    std::size_t N;
    switch (T) {
    case typename{U}[N]:
        return U*;
    default:
        fail("T is neither a container type nor an array type.");
    }
}
```

```

    }
}

```

The pattern-matching in the above example is useful but is more interesting in a more complex example. Let's define a codomain type function (the codomain is the return type of a mathematical function).

The algorithm is:

```

For a function object type F,
if F is a native function type, pointer to function type or pointer to member function type
    return the codomain type obtained by pattern-matching
else if F has an inner `codomain` typedef
    return it
else if F has a unique (non-overloaded, non-template) `operator()`
    return the codomain type obtained by pattern-matching the corresponding
        pointer-to-member function type
else
    fail

```

Note that for this algorithm to succeed, the function object type must correspond a single mathematical function. That is, it must be associated with a specific domain (argument types) and a specific codomain (return type). This excludes function objects types that correspond to a family of mathematical functions, such as user-defined function object types whose `operator()` is overloaded or template.

Here is an implementation with a type function:

```

typename CodomainType(FunctionObject F) { // FunctionObject is a concept
    typename Ret; // We declare an unconstrained type
    Class C; // and a class type (`Class` is a concept)
           // for the following pattern matching:

    switch (F) {
    case Ret (...): // function
    case Ret (*)(...): // pointer to function
    case Ret (C::*)(...): // pointer to member function
    case Ret (C::*)(...) const: // pointer to const member function
    case Ret (C::*)(...) volatile: // pointer to volatile member function
    case Ret (C::*)(...) const volatile: // pointer to const volatile member function
                                           // lvalue-ref, rvalue-ref, noexcept variants omitted.

        return RemoveCv(RemoveReference(Ret));
    default: // user-defined type
        // If there is a `codomain` inner typedef, return it.
        if (requires { typename F::codomain; }) {
            return F::codomain;
        }
        // Otherwise try to get a pointer to `operator()`.
        else if (requires { &F::operator(); }) {
            return CodomainType(decltype(&F::operator()));
        }
        fail("operator() must not be overloaded or template.");
    }
}

```

This example shows how pattern-matching, concepts, requires clause and if play nicely together in the context of type functions.

It is particularly useful to be able to regroup cases, in contrast to having an implementation scattered over multiple overloads.

This implementation shows that type functions allow a good expressivity, especially compared to the following C++17 metaprogramming equivalent:

```

template<typename T>
class has_nested_codomain_type {
    template<typename U>

```

```

    static std::true_type test(typename U::codomain_type*);

    template<typename>
    static std::false_type test(...);
public:
    using type = decltype(test<T>(nullptr));
};

template<typename T>
using has_nested_codomain_type_t = typename has_nested_codomain_type<T>::type;

template<typename T>
struct codomain_transform_arg : std::decay<T> {};

template<typename T>
struct codomain_class_pointer_helper;

template<typename Ret, typename C, typename... T>
struct codomain_class_pointer_helper<Ret (C::*)(T...)>
    : codomain_transform_arg<Ret> {};

template<typename Ret, typename C, typename... T>
struct codomain_class_pointer_helper<Ret (C::*)(T...) const>
    : codomain_transform_arg<Ret> {};

template<typename Ret, typename C, typename... T>
struct codomain_class_pointer_helper<Ret (C::*)(T...) volatile>
    : codomain_transform_arg<Ret> {};

template<typename Ret, typename C, typename... T>
struct codomain_class_pointer_helper<Ret (C::*)(T...) const volatile>
    : codomain_transform_arg<Ret> {};

template<typename T>
struct codomain_class_pointer : codomain_class_pointer_helper<decltype(&T::operator())> {};

template<typename T, typename HasNestedCodomainType>
struct codomain_class_dispatch {
    using type = typename T::codomain_type;
};

template<typename T>
struct codomain_class_dispatch<T, std::false_type> : codomain_class_pointer<T> {};

template<typename T>
struct codomain_class : codomain_class_dispatch<T, has_nested_codomain_type_t<T>> {};

template<typename T>
struct codomain_function;

template<typename Ret, typename ...T>
struct codomain_function<Ret (T...)> : codomain_transform_arg<Ret> {};

template<typename Ret, typename ...T>
struct codomain_function<Ret (*)(T...)> : codomain_transform_arg<Ret> {};

template<typename F, typename IsClass>
struct codomain_dispatch : codomain_class<F> {};

template<typename F>
struct codomain_dispatch<F, std::false_type> : codomain_function<F> {};

template<typename F>
struct codomain_type : codomain_dispatch<F, typename std::is_class<F>::type> {};

template<typename F>
using CodomainType = typename codomain_type<F>::type;

```

We used `RemoveReference` in `CodomainType`. Its implementation can also be a good example of pattern-matching:

```
typename RemoveReference(typename T) {
    typename U;
    switch (T) {
    case U&:
    case U&&:
        return U;
    default:
        return T;
    }
}
```

Finally, in `CodomainType` we saw an example of type function "chaining":

```
RemoveCv(RemoveReference(Ret))
```

We will see in [section 9](#) how to easily compose functions.

7. Processing sequence of types with for

Up to this point, we've seen overloaded type functions, with conditional and pattern-matching.

What if we want to process a sequence of types? For example, we could want to compute the common type of a sequence of types:

```
// Default version: n arguments (n > 0)
typename CommonType(typename T, typename... Ts) {
    // iterate over a pack of types
    // `for...` is implied because we are in a type function and operating on a
    // parameter pack.
    // This follows the same logic as `if` in type functions being an `if constexpr`.
    for (typename U: Ts)
        T = CommonType(T, U);
    return T;
}

// Specialization: 1 argument
typename CommonType(typename T) {
    return T;
}

// Specialization: 2 arguments
typename CommonType(typename T, typename U) {
    ...
}
```

This implies the possibility to iterate over a pack of types. Also, types are passed "by value" and manipulated as values. Above, `T` is only modified in the type function, not on call site.

An application example of the above code is:

```
template<typename... Args>
CommonType(Args...) compute(std::tuple<Args...> const& t);
```

In a similar way, we could write a type function returning the type with the biggest size. This shows the use of an integral value:

```
typename BiggestType(typename T, typename... Ts) {
    std::size_t N = sizeof(T);
    for (typename U: Ts) {
        if (sizeof(U) > N) {
            T = U;
        }
    }
}
```

```

        N = sizeof(T);
    }
}
return T;
}

// In another context, we can now write:
typename T = BiggestType(Args...);

```

Both `CommonType` and `BiggestType` are examples of reductions. We will see more on this in [operator overloading section](#).

8. ADL

We propose that type functions follow ADL, as shows the following example.

Sometimes, we don't want to provide `operator<` for a type because there is no natural total ordering. But we still want our type to be usable as a key of an associative container (`std::map`, `std::set`...). In this case, we want to specialize `std::less`.

Consider:

```

// Defined in a third-party library.
namespace game {
    class player {
        string name;
        int score;
    public:
        // No natural total ordering: should we first order on name and then on
        // score, or the opposite? Let's not provide `operator<` and let the
        // user decide. But we still want this type to be easily usable
        // as an associative container key.
    };
} // So we have to close our namespace...

// Reopen std...
namespace std {
    // ...and add our `less` specialization
    template<>
    struct less<game::player> {
        constexpr bool operator()(game::player const& a, game::player const& b) const {
            return a.name < b.name || (!(b.name < a.name) && a.score < b.score);
            // Note: We could also order first on the score to be more efficient.
        }
    };
}

// Now we close `std` and reopen `game` to continue our work.
namespace game {
    // ...
}

```

Having to reopen namespace `std` to put our code inside it is ugly and verbose.

Consider now the type function alternative with ADL:

```

namespace std {
    // Slight modification of `set`: it now uses the type function `Less`
    // to get the `Compare` type.
    // Note: for the sake of simplicity, this example ignores the allocator type.
    template<typename Key, typename Compare = Less(Key)>
    class set;

    // General definition:

```

```

// `Less` takes a totally ordered type and return a binary predicate type.
BinaryPredicate Less(TotallyOrdered T) {
    // We return an anonymous type.
    return struct {
        constexpr bool operator()(T const& a, T const& b) const {
            return a < b;
        }
    };
}

// In our code:
namespace game {
    class player {
        // ...
    };

    // We specialize `Less` for the type `player`.
    // See explanations below on the `requires` clause and the use of `==`.
    BinaryPredicate Less(typename T) requires(T == player) {
        return struct {
            constexpr bool operator()(T const& a, T const& b) const {
                return a.name < b.name || (!(b.name < a.name) && a.score < b.score);
            }
        };
    }
}

// In yet another namespace:
namespace xyz {
    void my_function() {
        // Will use the `Less` version from namespace `game` as expected.
        std::set<game::player> players;
    }
}

```

With type functions we don't have to reopen namespace `std` anymore and the mechanism to find the right overload (ADL) is well-known. This results in shorter and clearer code. The previous example is built around `std::less`, but another good example would be with `std::hash`.

The `Less` specialization for type `player` uses a `requires` clause to restrict this overload the `player` type only. This relies on `==` which is defined on types with the expected semantics (its external behavior is identical to `std::is_same_v`). This shows that having operators defined on types is desirable. See [section 10](#) for more examples.

In a previous example (see [section 4](#)), we did not use a `requires` clause with `==` to restrict a type function specialization to one type, but we used instead the `concept()` builtin function. These two solutions are here equivalent:

```

// `requires` + `==` solution:
BinaryPredicate Less(typename T) requires(T == player) {
    return struct {
        constexpr bool operator()(T const& a, T const& b) const {
            return a.name < b.name || (!(b.name < a.name) && a.score < b.score);
        }
    };
}

// equivalent to `concept()` solution:
BinaryPredicate Less(concept(player) T) {
    return struct {
        constexpr bool operator()(T const& a, T const& b) const {
            return a.name < b.name || (!(b.name < a.name) && a.score < b.score);
        }
    };
}

```


9. Composition

We've seen in [section 6](#) an example of chaining type functions:

```
RemoveCv(RemoveReference(T))
```

We can write a type function to compose `RemoveCv` and `RemoveReference`:

```
typename RemoveCvReference(typename T) {  
    return RemoveCv(RemoveReference(T));  
}
```

Also, assuming `RemoveCv` and `RemoveReference` also remove cv-qualifiers and reference qualifiers from member functions (not in the standard), we can simplify `CodomainType` by reducing the pattern-matching to these cases:

```
typename CodomainType(FunctionObject F) { // `FunctionObject` is a concept  
    ...  
    switch (RemoveCvReference(F)) {  
        case Ret (...): // function  
        case Ret (*)(...): // pointer to function  
        case Ret (C::*)(...): // pointer to member function  
                                // (noexcept variants omitted)  
        return RemoveCvReference(Ret);  
        default: // user-defined type  
        ...  
    }  
}
```

Since we compose types with (type) functions, we can define a composition type function. For that, we define a `TypeFunction` concept:

```
// A type function takes a type and returns a type.  
template<typename F>  
concept TypeFunction = requires(typename... Ts) { // new syntax (see below).  
    { F(Ts...) } -> typename;  
};
```

This implies an adaptation of C++20 concepts to introduce types in `requires` clause. The exact impact of this change is to be determined.

Every type function we defined so far models of course the `TypeFunction` concept.

We also propose that lambda syntax is extended to operate on types: when operating on types, the constructed closure is a type function.

With this, we can write:

```
// The domain of `G` must match the codomain of `F`.  
TypeFunction Compose(TypeFunction G, TypeFunction F) {  
    // A lambda operating on types is a type function.  
    // Notice the parallel with runtime programming.  
    return [=](typename... Ts) { // the only allowed capture mode is by value.  
        return G(F(Ts...));  
    };  
}
```

We can now rewrite `RemoveCvReference` in this way:

```
typename RemoveCvReference = Compose(RemoveCv, RemoveReference);
```

And possibly use it in other compositions:

```
// `CodomainOfValueType` algorithm:
// - first removes cv and reference qualifiers
// - then takes the value type
// - then takes the codomain type
// This makes sense for example if the input type is a
// (possibly reference to cv-qualified) container of functions.
typename CodomainOfValueType = Compose(Compose(CodomainType, ValueType), RemoveCvReference);
```

See [section 10. d](#) for an alternative syntax. The ability to perform simple compositions opens the door to more complex composition, as explained in [section III. 1](#).

10. Operator overloading

a. Comparison operators

We propose that == and != are defined on types with the expected semantics (A == B gives the same result as std::is_same_v<A, B>). These operators allow to express much more clearly the intention of the user.

Compare:

```
// constructor of `my_type`
// We prevent it to "swallow" the copy constructor
// (simplification: doesn't handle derived types).
template<typename T,
    typename = std::enable_if_t<std::negation_v<std::is_same<std::decay_t<T>, my_type>>>>
    my_type(T&& t);
```

with

```
template<typename T, typename = std::enable_if_t<std::decay_t<T> != my_type>>
    my_type(T&& t);
```

or if we assume standard type function equivalents:

```
template<typename T, typename = std::EnableIf(std::Decay(T) != my_type)>
    my_type(T&& t);
```

Another use of == has already been seen in [section 8](#) to restrict a type function to a single type.

It is also useful with respect to void: as long as void is an exceptional type in the type system, handling it requires special cases (e.g. std::future<void>, composing void (T) with other functions, etc.):

```
typename MyTypeFunction(typename T) {
    if (T == void) { // special case
        ...
    } else {
        ...
    }
}
```

b. Relational operators

It is also possible to define a total ordering on types by using their type index (assuming type index comparisons are constexpr).

First, we define a type attribute that associates a type to its index (more on type attributes in [section 12. a](#)):

```
std::type_index indexof(typename T) {
    return std::type_index(typeid(T));
}
```

Then we can define relational operators:

```
bool operator<(typename A, typename B) {
    return indexof(A) < indexof(B);
}

bool operator>(typename A, typename B) {
    return B < A;
}

bool operator<=(typename A, typename B) {
    return !(B < A)
}

bool operator>=(typename A, typename B) {
    return !(A < B)
}
```

This definition of < is indeed a total ordering: it is associative and it respects the trichotomy law (only one of the following holds: $A < B$, $B < A$, or $A == B$).

It is also possible to define weak orderings (meaning, in the trichotomy law equality is replaced by an equivalence) on types by using `sizeof(T)` or `alignof(T)`.

A use example of < is given in the [section on sum and product types](#).

c. Logical operators

We can also overload operators by concept. For example, it makes sense to define ! on predicate types as returning the complement type. In the following example, we want to keep all characteristics of the input predicate type, only negating the function call. For that, we use reflection and code injection (see [\[P0707\]](#)):

```
Predicate ComplementType(Predicate P) requires Class(P) {
    // We inject code in P (variant of p0707 syntax)
    // Note that types are passed "by value", so we are not
    // modifying the original predicate.
    // The algorithm is:
    //     for each operator()
    //         change its signature by adding a dummy int parameter
    //         make it private
    //         add an operator() with the original signature (i.e. without int)
    //         make it return the complement of the original operator()
    return ->(P) {constexpr {
        for (auto f: $P.functions()) {
            if (f.name == "operator()") {
                // Change the signature of the original `operator()` to
                // leave room for the new one.
                f.parameters().push_back($int);
                f.make_private();

                // Inject a new `operator()` negating the original one.
                -> {
                    bool operator()(f.parameters())$ f.qualifiers()$ {
                        // Call the original version.
                        constexpr {
                            f.parameters().pop_back();
                            -> { return ! operator()(f.parameter_names()$, 0); }
                        }
                    }
                }
            }
        }
    }
};
```

```
// Simply forward to `ComplementType`
Predicate operator!(Predicate P) requires Class(P) {
    return ComplementType(P);
}
```

Now we can negate a predicate type. For example:

```
typename greater_eq = !Less(int);
```

Also, in the definition of ComplementType, \$P.functions() is really a type attribute (see [section 12. a](#)) and could be written more consistently functions(P):

```
for (auto f: functions(P)) {
    if (f.name == "operator()") {
        ... // idem
    }
}
```

Finally, the type manipulated by operator! must be a Predicate type and a Class type. We'll see in [section III 1.](#) a more powerful way to compose concepts.

We can also define other boolean operators, such as &&:

```
Predicate ConjunctionType(Predicate P0, Predicate P1) {
    // Here, we don't inject code. We just embed the two predicates
    // into a new predicate type:
    return
        class conj {
            P0 p0;
            P1 p1;
        public:
            // A predicate type is regular:
            friend bool operator==(conj const& x, conj const& y) {
                return x.p0 == y.p0 && x.p1 == y.p1;
            }
            // A predicate type doesn't modify its argument.
            template<typename... Args>
            bool operator()(Args const&... args) {
                return p0(args...) && p1(args...);
            }
            // `const` version
            template<typename... Args>
            bool operator()(Args const&... args) const {
                return p0(args...) && p1(args...);
            }
        };
};

// Simply forward to `ConjunctionType`
Predicate operator&&(Predicate P0, Predicate P1) {
    return ConjunctionType(P0, P1);
}
```

Note: ComplementType and ConjunctionType are indeed type constructors: they take a type and create a new type (see [section 12. b](#)). A unary type constructor (such as ComplementType) is similar to a [\[P0707\]](#)'s metaclass (see [section 11](#)). But a n-ary type constructor (such as ConjunctionType) cannot *a priori* be made with a metaclass.

We can now use && to create new predicates:

```
// `is_male_t`, `has_brother_t` and `has_sister_t` are predicates.
Predicate is_only_son_t = is_male_t && !has_brother_t && !has_sister_t;

// Compare the readability of the previous form with this one:
Predicate is_only_son_t = ConjunctionType(ConjunctionType(is_male_t,
    ComplementType(has_brother_t)), ComplementType(has_sister_t));
```

```

// Or if we only had object functions (i.e. functions on objects),
// assuming `&&` and `!` had been overloaded accordingly:
Predicate is_only_son_t = Decay(decltype(
    std::declval<is_male_t>()
    && !std::declval<has_brother_t>()
    && !std::declval<has_sister_t>()
));

```

Having type function operators is readable and non-ambiguous.

Some use of conjunction and negation are common so it is useful to define type functions such as `EquivalenceType`:

```

// A relation is a homogeneous binary predicate
// (i.e. a predicate on two elements of the same type).
// Precondition: R is a weak ordering
Relation EquivalenceType(Relation R) {
    // Assume the existence of `ConverseType` that swaps
    // the order of the two parameters of a relation type:
    // with a relation r, the converse of r(a, b) is r(b, a).
    // The returned relation is true for two values if none
    // is "less" than the other one, which forms an equivalence.
    return !R && !ConverseType(R);
}

```

We can use it for example to transform a total ordering into an equivalence and compare sequence of elements:

```

// We want to know if two strings are equivalent when we ignore case and accents.
// `is_char_before_nocase_noaccent_t` is a weak ordering.
// E.g. with `before` an instance of this type, the following are true:
// before('a', 'b'), before('b', 'c'), before('a', 'c'),
// !before('a', 'a'), !before('b', 'a'), !before('c', 'a'),
// !before('A', 'a'), !before('a', 'A'), !before('a', 'à'), !before('à', 'a'),
// !before('â', 'a'), !before('a', 'â'), etc.

std::string s0 = ...;
std::string s1 = ...;

// `lexicographical_equivalent` requires an equivalence.
// For a definition of `lexicographical_equivalent`, see
// `Elements of Programming`, section 7.4.
if (lexicographical_equivalent(begin(s0), end(s0), begin(s1), end(s1),
    EquivalenceType(is_char_before_nocase_noaccent_t){})) {
    ...
}

```

d. Composition operator

In [section 9](#), we composed type function with this notation:

```

typename CodomainOfValueType = Compose(Compose(CodomainType, ValueType), RemoveCvReference);

```

which has the form:

```

// x is a function that first applies f, then g on the result of f, then h on the result of g.
x = compose(h, compose(g, f));

```

The longer the chain of composition, the less readable this notation is, and a linear notation such as the following one could be preferable:

```

// x is a function that first applies f, then g on the result of f, then h on the result of g.
x = f | g | h;

```

So we define `|` for type functions:

```
TypeFunction operator|(TypeFunction F, TypeFunction G) {
    return Compose(G, F);
}
```

Now we can rewrite

```
typename CodomainOfValueType = Compose(Compose(CodomainType, ValueType), RemoveCvReference);
```

as:

```
typename CodomainOfValueType = RemoveCvReference | ValueType | CodomainType;

// C is a container of function objects
template<Container C>
auto do_stuff(C&& functions) {
    std::vector<CodomainOfValueType(C)> results;
    // ...
}
```

or directly in-place for a one-shot use:

```
// C is a container of function objects
std::vector<(RemoveCvReference | ValueType | CodomainType)(C)> results;
```

We will see more complex compositions in the [section III 1](#).

e. Sum types and product types

On a more theoretical side, there exists an algebra of types that defines sum and product on types. Sum corresponds to union (or variant) and product to struct (or tuple). There is a special type that behaves as the 0 of the sum, and another one that behaves as the 1 of the multiplication. Also, the product distributes over the sum. There is also exponentiation that corresponds to function types, but this is not considered in the following examples for the sake of simplicity.

The natural way to express sum types and product types is to use operators + and *. As before, we first define type functions and then implement operators as simply forwarding to these type functions.

```
// We want to avoid nesting to make the operation associative.
// E.g. `OpSumType(OpSumType(int, bool), char)` is the type
// `variant<int, bool, char>` instead of `variant<variant<int, bool>, char>`.
// Similarly, `OpSumType(int, OpSumType(bool, char))` is the type
// `variant<int, bool, char>` instead of `variant<int, variant<bool, char>>`.
typename OpSumType(typename A, typename B) {
    switch (B) {
        case std::variant<typename...{ArgsB}>:
            return detail::OpSumType(A, ArgsB...);
        default:
            return detail::OpSumType(A, B);
    }
}

namespace detail {
    typename OpSumType(typename A, typename... ArgsB) {
        switch (A) {
            case std::variant<typename...{ArgsA}>:
                return std::variant<ArgsA..., ArgsB...>;
            default:
                return std::variant<A, ArgsB...>;
        }
    }
}

typename operator+(typename A, typename B) {
    return OpSumType(A, B);
}
```

```

}

// `operator*` is similar to `operator+` but is implemented with a
// `OpProductType` type function that returns a `std::tuple` type instead
// of a `std::variant` type.
// Thus, `OpProductType(int, bool)` is the type `tuple<int, bool>` and both
// `OpProductType(OpProductType(int, bool), char)` and
// `OpProductType(int, OpProductType(bool, char))` are the type `tuple<int, bool, char>`.

typename operator*(typename A, typename B) {
    return OpProductType(A, B);
}

```

Note that in the previous code, [P0095](#)'s `lvariant` could be used instead of `std::variant` for an alternative implementation.

Also, an alternative to the previous implementation of `OpSumType` and `OpProductType` using a `switch case` is to do pattern matching is to specialize functions, but it might be more cumbersome:

```

typename OpSumType(typename A, typename B);

typename OpSumType(concept(std::variant<typename...{ArgsA}>) A, typename B);

typename OpSumType(typename A, concept(std::variant<typename...{ArgsB}>) B);

typename OpSumType(concept(std::variant<typename...{ArgsA}>) A,
                    concept(std::variant<typename...{ArgsB}>) B);

// Idem for `OpProductType` with `std::tuple` instead of `std::variant`.

```

We can now use these operators to define algebraic datatypes:

```

typename name_or_id_t = std::string + int;
name_or_id_t n0{"bob783"};
name_or_id_t n1{95248};

typename name_and_score_t = std::string * int;
name_and_score_t n2{"bob783", 10000};

// `uninstanciable_t` is roughly the `0` of the sum.
// Because it cannot be instanciated, it does not add real information
// to the type.
struct uninstanciable_t {
    uninstanciable_t() = delete;
};

typename name_t = std::string + uninstanciable_t;
name_t n3{"bob"};

// std::monostate is roughly the `1` of the product because it does not
// add any real information to the type.
typename name_t = std::string * std::monostate;
name_t n4{"bob"};

// Here is an example of distributivity of the product over the sum.
// A "registered player" has a status. He also has either name or an id:
typename registered_player_0_t = status_t * (std::string + int);

// Equivalent to:
typename registered_player_1_t = (status_t * std::string) + (status_t * int);

// Note that `registered_player_0_t` is not the same type as `registered_player_1_t`.
// But they are equivalent (isomorphic) in the sense that it is possible to convert
// from one to the other and convert back without "losing" any information.

```

We can have arbitrary long chains of `+` or `*`, and performing a sum or a product is a special case of reduction:

```

typename ReduceType(TypeFunction Op, typename T, typename... Ts) {
    for (typename U: Ts)
        T = Op(T, U);
    return T;
}

typename SumType(typename T, typename... Ts) {
    // Reduce in terms of the binary version:
    return ReduceType(OpSumType, T, Ts...);
}

typename ProductType(typename T, typename... Ts) {
    // Reduce in terms of the binary version:
    return ReduceType(OpProductType, T, Ts...);
}

```

In section about [for applied to sequence of types](#), we gave as an example the type functions BiggestType and CommonType. They are also special cases of reduction and can be reimplemented this way:

```

typename CommonType(typename T, typename... Ts) {
    TypeFunction Op = [](typename A, typename B) {
        ...
    };
    return ReduceType(Op, T, Ts...);
}

typename BiggestType(typename T, typename... Ts) {
    TypeFunction Op = [](typename A, typename B) {
        if (sizeof(B) > sizeof(A)) return B;
        else return A;
    };
    return ReduceType(Op, T, Ts...);
}

```

11. Metaclasses

a. Metaclasses as type functions

A metaclass (as described in [P0707]) is primarily a mechanism to generate new types. Type functions are rather a mechanism to associate types. But coupled with other mechanisms such as code injection, type functions can also generate new types in much the same way as metaclasses.

Also from a logical point of view, a metaclass can be viewed as a function associating an input type (the "prototype class") to a new type (the generated type).

For example, the metaclass interface used in

```

interface Shape {
    int area() const;
    void scale_by(double factor);
};

```

is expected to approximatively generate this code:

```

// In an unspecified and unique namespace:
namespace __xyz {
    struct prototype_Shape {
        int area() const;
        void scale_by(double factor);
    };
}
/* apply `interface` on `__xyz::prototype_Shape` */;

```


This is equivalent to the following code with interface being now an equivalent type function:

```
typename Shape = interface(  
    struct { // no need to name the struct  
        int area() const;  
        void scale_by(double factor);  
    }  
);
```

In [P0707], composing metaclass is done in an imperative way:

```
// `value` and `serializable` are metaclasses.  
constexpr void serializable_value(meta::type target, const meta::type source) {  
    value(target, source);  
    serializable(target, source);  
};
```

With type functions, composition is simply function composition:

```
// `value` and `serializable` are now type functions equivalent  
// to the previous metaclasses.  
typename point = serializable(value(  
    struct {  
        int x;  
        int y;  
    }  
));
```

Also, note that a metaclass takes an "in/out" argument (the target) and return void. This makes a metaclass different from a mathematical function, preventing the usual function composition and more advanced compositions (see [section III 1](#)).

If we want to give a name to our previous composition, we can write:

```
typename serializable_value = value | serializable;  
  
typename point = serializable_value(  
    struct {  
        int x;  
        int y;  
    }  
);
```

Thus, we have a unified way to perform composition.

We can also ensure at different levels that the returned type models some concept:

- in the definition of a type function, e.g.:

```
Regular value(typename T); // the type returned by `value` models the `Regular` concept
```

- on call site:

```
Regular point = serializable_value(  
    struct {  
        int x;  
        int y;  
    }  
);
```

As a side note, the notation

```
serializable_value point {  
    int x;
```

```
    int y;
};
```

could be introduced as syntactic sugar for

```
typename point = serializable_value(
    struct {
        int x;
        int y;
    }
);
```

Thus, it seems metaclasses are only a specific case of type functions (with potential extra syntactic sugar).

Now if we use the same code injection mechanism as in [\[P0707\]](#), we can write for example:

```
// `value` is a type function that adds a memberwise `operator==` if it doesn't
// already exist (this is a simplified version for the sake of example).
typename value(typename T) {
    // Inject code in T, creating a new type (the input type is not modified):
    return ->(T) {
        constexpr {
            // If there is no `operator==` defined:
            if (! requires(T a) { a == a; }) -> {
                friend bool operator==(T const& a, T const& b) {
                    constexpr {
                        for (auto v: variables(T)) // or $T.variables()
                            -> { if (! (a.(v.name)$ == b.(v.name)$)) return false; }
                    }
                    return true;
                }
            }
        }
        // ...
    };
}
```

Since types are passed "by value" to type functions, the code is injected in a copy of T and the original T is not modified. Injecting in the copy of an existing type is interesting if we only want to add methods and members, or if we want to add checks.

If we prefer to start from scratch by injecting in an empty type, we can write:

```
typename value(typename T) {
    typename empty = struct {};
    return ->(empty) {
        // fill `empty`

        // Because `struct` was used, everything is public by default.

        // We could also have written `typename empty = class {}`
        // to have everything private by default.
    };
}
```

Or if we don't need to name the type:

```
typename value(typename T) {
    return ->(struct {}) { // or ->(class {})
        // ...
    };
}
```

In fact, any expression yielding a type is admissible:

```
// A value is a special kind of basic_value.
typename value(typename T) {
    return ->(basic_value(struct {})) {
        // ...
    };
}
```

or leveraging operators:

```
typename my_variant_value(typename T, typename U) {
    if (T == void || U == void) {
        ...
    } else {
        return ->(value(T) + value(U)) {
            ...
        };
    }
}
```

Finally, it is also possible to fill the returned type in a more imperative manner:

```
typename value(typename T) {
    typename Res = struct {};
    for (auto m: members_and_bases(T))
        ->(Res) m;
    // ...
    return Res;
}
```

[b. .is](#)

In [P0707], $\$T.is(M)$ is used to determine if the type T satisfies the requirements of the metaclass M . It is a predicate and has roughly the same role as the present proposal's writing $C(T)$ where C is a concept and T a type. A metaclass can also be used instead of a concept in a template parameter introduction.

Having metaclasses acting as concepts ends up having in the language two mechanisms with overlapping responsibilities. We think this is not sound and would rather keep concepts only and add a mechanism to generate a concept from a type function. The algorithm for this would be the same as for metaclasses. With a type T and a type function F , a concept generated from F would evaluate to true if and only if:

1. applying F to T succeeds; and
2. the resulting type has no new members not already present in T .

Syntactically, we can reuse the concept built-in function. Previously, we used it to generate a concept modeled by a unique type. Now we use it to generate a concept from a type function:

```
template<concept(value) T> // `T` "is" a `value`
T my_function(T t) {
    ...
}
```

Therefore, the features of `.is` can also be brought by type functions with the benefit of having roles soundly kept separated, with non-overlapping mechanisms (types, type functions, concepts) and non-ambiguous code.

[c. .as](#)

In [P0707], $\$T.as(M)$ is used to apply the metaclass M on the type T . This is what simply corresponds in the present proposal to a type function application. For example:

```
// P0707: transforming the type `legacy_point` with the `ordered` metaclass
using ordered_point = $legacy_point.as(ordered);
```

corresponds in terms of type functions to:

```
// The present proposal  
typename ordered_point = ordered(legacy_point);
```

We don't need a special syntax. The writing is intuitive as it is simply function application.

The trick of `.as` used for 'strong typedef' can also be done with type functions:

```
// P0707: using an empty metaclass for 'strong typedef'  
constexpr void new_type(meta::type target, const meta::type) {}; // no-op metaclass fn  
using my_T = $T.as(new_type); // my_T is a 'strong typedef' of T  
using handle = $int.as(new_type);  
using score = $unsigned.as(new_type);  
using player = $string.as(new_type);
```

corresponds in terms of type functions to:

```
typename new_type(typename T) {  
    // We inject no code into T, resulting in the creation of a new type  
    // similar to the original one. The logic is the same as the empty  
    // metaclass function trick.  
    return ->(T) {};  
}  
  
typename my_T = new_type(T);  
typename handle = new_type(int);  
typename score = new_type(unsigned);  
typename player = new_type(string);
```

So it seems type functions, when coupled to an injection mechanism, are a superset of metaclasses. By manipulating types directly we believe the resulting code is clearer and more consistent with the rest of the language.

12. Type attributes and type constructors

a. Type attributes

In [section I. 3](#), we classified type functions in three categories: type functions, type attributes and type constructors. Up to this point, we have almost only talked about type functions.

A type attribute associates a type to a compile-time value. For example for a type `T`, `sizeof(T)` and `alignof(T)` are built-in type attributes: they both return an unsigned integral value that describes a characteristic of the type. Notice that `sizeof(T)` and `alignof(T)` already use a function syntax.

Other generally useful type attributes could be:

- `nameof(T)`: returns a string naming the type
- `membersof(T)`: returns the members of a type
- `arityof(F)`: returns the number of arguments of a function object type
- etc.

The definition of a type attribute is the same as a type function, except it returns a value instead of a type:

```
// Binary operation that blends two colors.  
class BlendColors {  
    ...  
    Color operator()(Color const& a, Color const& b) const;  
    ...  
};  
  
// `arityof` should be a built-in type attribute.
```

```
// This is simply an example to show the definition of a type attribute.
std::size_t arityof(concept(BlendColors) F) {
    return 2u;
}
```

b. Type constructors

Type functions associate a type to an affiliated type. For example `IteratorType(T)` associates `T` to its iterator type. In contrast, a type constructor creates a new type. For example, the built-in "pointer-to" type constructor is used by appending a `*` to a type (e.g., `T*` for `T`), thus creating a new type.

- `*`, `&`, `const` can be viewed as unary type constructors (e.g., `int*`, `int&`, `int const`)
- `[]` can be viewed as a binary type constructor (e.g. `int [5]`).
- `()` can be viewed as a n-ary type constructor (e.g. `bool (int, char)`)
- `struct` and `class` can be viewed as n-ary type constructors (e.g. `struct {int i; bool b;}`)

Operators we defined in [section 10. c](#) were also type constructors:

- `!` (unary, see `ComplementType`)
- `&&` (binary, see `ConjunctionType`)
- etc.

Every class template is a type constructor:

- `std::vector` (binary, e.g. `std::vector<int>`, `std::vector<float, MyAlloc>`)
- `std::function` (n-ary, e.g. `std::function<bool (int, char)>`)
- `std::tuple` (n-ary, e.g. `std::tuple<int, double, float>`)
- `std::variant` (n-ary, e.g. `std::variant<int, double, float>`)
- etc.

This can be formulated as type functions:

```
typename Ptr(typename T) {
    return T*;
}

typename Vector(typename T, typename A = std::allocator<T>) {
    return std::vector<T, A>;
}

typename Tuple(typename... Ts) {
    return std::tuple<Ts...>;
}

// etc.
```

Unifying syntax allows to reusing our composition tools:

```
typename VecOfPtrTuple = Tuple | Ptr | Vector;

typename V = VecOfPtrTuple(int, bool);
static_assert(V == std::vector<std::tuple<int, bool>*>);
```

The difference between type constructors and type functions being semantic, they can be freely mixed:

```
typename I = IteratorType | VecOfPtrTuple;
static_assert(I(int, bool) == std::vector<std::tuple<int, bool>*>::iterator);
```

13. Note on the implementation

This section gives hints to speed up possible implementations. It shows that type functions may be implemented by generating code in terms of other (ongoing) proposals.

The fact that the above examples often look like template specializations doesn't mean that a type function has to be implemented in terms of templates. It may seem more natural to implement it in terms of a constexpr function. For example, the following code where C is a container type:

```
IteratorType(C) it;
```

could result in this generated code (assuming [\[P0425\]](#) features):

```
TYPENAME(__IteratorType(REFLEXPR(C))) it;
```

The default version of __IteratorType could be roughly implemented in this way:

```
std::meta::type __IteratorType(std::meta::type ContainerType) {  
    // assuming this notation is valid  
    return ContainerType.inner_typedefs["iterator"];  
}
```

Or (assuming [\[P0707\]](#)'s metaclasses), maybe something like:

```
__IteratorType($C)$ it;
```

where __IteratorType is a constexpr function.

The type function IteratorType we previously defined this way:

```
ForwardIterator IteratorType(typename T) {  
    if (Container(T))  
        return T::iterator;  
    typename U; // declare an unconstrained type  
    std::size_t N; // declare an integral constant  
                  // for the following pattern-matching.  
    switch (T) {  
    case U[N]:  
        return U*;  
    default:  
        fail("T is neither a container type nor an array type.");  
    }  
}
```

could lead to this code generation:

```
constexpr void __IteratorType(meta::type target, const meta::type source) {  
    if (is_container(t))  
        target = t.type("iterator");  
    if (is_array(t))  
        target = make_pointer(element_type(t));  
    fail("The type is neither a container type nor an array type.");  
}
```

Nevertheless, if we have type function overloads, e.g. one IteratorType for container types and another one for array types, having a unique meta-type (meta::type) is not enough. It seems one meta-type per concept would be needed, so that it would be possible to write:

```
// Container type overload  
meta::iterator_type __IteratorType(meta::container_type t) {  
    return t.type("iterator");  
}  
  
// Array type overload  
meta::iterator_type __IteratorType(meta::array_type t) {
```

```

    return make_pointer(element_type(t));
}

```

Of course, it is also technically possible to generate classical template code:

```

namespace detail {
    // Default case.
    template<typename T>
    struct __IteratorTypeSwitch {
        // T is neither a container type nor an array type.
    };

    // Array type case.
    template<typename U, std::size_t N>
    struct __IteratorTypeSwitch<U [N]> {
        using type = U*;
    };

    // `T` is a container type.
    template<typename T, bool isContainer>
    struct __IteratorTypeIfContainer {
        using type = typename T::iterator;
    };

    // `T` is _not_ a container type. Continue with the `switch`.
    template<typename T>
    struct __IteratorTypeIfContainer<T, false> : __IteratorTypeSwitch<T> {
    };
} // namespace detail

// Uses `Container` concept.
template<typename T>
struct __IteratorType : detail::__IteratorTypeIfContainer<T, Container<T>()> {
};

```

Generating code based on meta-types as shown above may be preferable as it avoids template instantiations. Even in this case, type functions allows to:

- avoid the reflexion / reification juggling
- unify syntax between object functions and type functions
- resulting in a simpler and more consistent language

See [introduction](#) for further details.

This also opens the door for further simplifications / unifications as will be shown in the following.

III. Concept functions

As language entities, concepts should also be manipulable. Below are several examples of concept functions.

1. Optional() and composition

Consider our previous implementation of `IteratorType`:

```

Iterator IteratorType(typename T) {
    if (Container(T)) // `Container` is a concept
        return T::iterator;
    else if (Array(T)) // `Array` is a concept
        return Decay(T);
    fail("T is neither a container nor an array.");
}

```

It will fail if τ is neither a container type nor an array type. This is not desirable in more advanced compositions as we will see below. To avoid failing, we can define a special `nil_t` type and return it as a fall-back. The problem is the type returned by `IteratorType` must be an iterator type and `nil_t` is not. To address this, we first define an `Optional` concept function with the following definition:

```
struct nil_t {
};

// Wrapper to perform specializations (see below).
template<typename T = nil_t>
struct opt_t {
};

// This is a concept function: it takes a concept and returns another concept.
concept Optional(concept C) {
    return
        template<typename T>
        concept OptionalC = requires(typename U) {
            requires T == opt_t<U> && (U == nil_t || C(U));
        };
}

// Alternative assuming it's possible to introduce in-place a type
// with the syntax `<concept-name>{<type-name>}`:
concept Optional(concept C) {
    return
        typename{T}
        concept OptionalC =
            T == opt_t<typename{U}> && (U == nil_t || C(U));
}
```

For a concept `C`, the concept returned by `Optional(C)` is modeled by all types equal to `opt_t` parametrized by `nil_t` or parametrized by a type modeling `C`.

As a side note, it would be nice to be able to omit the name of the returned concept (`OptionalC`) since it is not really needed, and simply write:

```
return
    typename{T}
    concept =
        T == opt_t<typename{U}> && (U == nil_t || C(U));
```

We can now write a total version of `IteratorType` (a total function is defined for every value of its domain, i.e. for any input argument):

```
Optional(Iterator) SafeIteratorType(typename T) {
    if (Container(T)) // `Container` is a concept
        return opt_t<T::iterator>;
    else if (Array(T)) // `Array` is a concept
        return opt_t<Decay(T)>;
    else
        return opt_t<>; // wraps `nil_t`
}
```

And rewrite `IteratorType` in terms of `SafeIteratorType`:

```
Iterator IteratorType(typename T) {
    switch (SafeIteratorType(T)) {
        case opt_t<typename{U}>: // introduce U here
            if (U != nil_t)
                return U;
            // go to default case
        default:
            fail("Couldn't get the iterator type for " +
                nameof(T)); // for `nameof`, see section 12. a
    }
}
```



```
    }
}
```

Let's assume we write `SafeCodomainType` and `SafeValueType` in the same way, that is with the following signatures:

```
Optional(typename) SafeCodomainType(typename T);
Optional(typename) SafeValueType(typename T);
```

Even if it is possible to compose non-safe versions with the default composition operator:

```
TypeFunction F = ValueType | CodomainType | IteratorType;
```

composing safe versions requires more work:

- if the result of the previous function is empty (i.e. `opt_t<>`), do not call the next function but directly return `opt_t<>`
- otherwise, extract the type parameter out of `opt_t` and pass it to the next function

We name this kind of compositions "k-compositions" ("k" for "kleisli") and express it with the following code:

```
// `G` and `F` are type functions taking a type and returning an `opt_t`
TypeFunction KCompose(TypeFunction G, TypeFunction F) {
    return [=](typename X) {
        // Apply `F`, and depending on the result apply `G` in a relevant way.
        return KBind(G, F(X));
    };
}

// This is the "k-bind" specialization for `opt_t`:
// `F` outputs `opt_t` but the type function `G` expects a non-`opt_t` type as input.
typename KBind(TypeFunction G, concept(opt_t<typename{Y}>) T) {
    if (Y == nil_t) {
        // The previous function produced no result, so we can't call G:
        // just return an empty `opt_t`.
        return opt_t<>;
    }
    // We have a result so we call `G`.
    return G(Y);
}

// A simple overload to make the code more readable.
TypeFunction operator>>(TypeFunction F, TypeFunction G) {
    return KCompose(G, F);
}
```

Now we can compose our safe type functions:

```
TypeFunction F = SafeValueType >> SafeCodomainType >> SafeIteratorType;
// Compare this with the simple composition version:
// TypeFunction F = ValueType | CodomainType | IteratorType;

static_assert(F(list<function<vector<int> ()>>) == opt_t<vector<int>::iterator>);

// Here, `SafeValueType` returns `opt_t<>` because a `std::function` has no value type.
static_assert(F(function<vector<int> ()>) == opt_t<>);

// Here, `SafeCodomainType` returns `opt_t<>` because a `int` has no codomain type.
static_assert(F(list<int>) == opt_t<>);

// Here, `SafeIteratorType` returns `opt_t<>` because an `int` has no iterator type.
static_assert(F(list<function<int ()>>) == opt_t<>);
```

2. Instrumented

When writing algorithms, it is good to test their complexity. We can for example check, for a given call, the number of comparisons.

```
// `value` is a value of type `T`.
// `tranfo` is a transformation on `T`, i.e. a function taking a `T` and returning a `T`.
// `is_defined` is a predicate on `T`, i.e. a function taking a `T` and returning a `bool`.
// (see Elements of Programming, 2.3 Collision Point)
assert(circular(value, tranfo, is_defined));

// Assert that equality has been called 4 times on `T`.
// `Operation(T)` is an enumeration type on operations available on `T`
// (equality, copy, assignment, etc.).
assert(count<T>(Operation(T)::equal) == 4);

// Reset the equality count.
count<T>(Operation(T)::equal) = 0;
```

If circular requires for value an object of a regular type, the above test requires in fact an "instrumented" regular type. "instrumented" means that T must have an Operation(T) enumeration type and a count function to query how many times a given operation has been performed.

Thus, any concept could be instrumented. We could have InstrumentedRegular, InstrumentedIterator (allowing to ask how many times an iterator has been incremented, for example), InstrumentedContainer (allowing to ask many times an element has been "pushed back", for example), and so on. Instrumented is really a concept function: it takes a concept and returns an enriched concept with additional constraints.

```
concept Instrumented(concept C) {
    return
        typename{T}
        concept InstrumentedC =
            C(T) // the type `T` must model the concept `C`
            && requires {
                // `T` must have an operation enum type
                typename Operation(T);
                requires Enumeration(Operation(T)); // assumes an `Enumeration` concept
            }
            && requires(Operation(T) op) {
                // must have a count method
                { count<T>(op) } -> int&;
            };
}
```

Now we can use Instrumented to transform concepts:

```
// Generic function to test a `copy_if` algorithm.
template<ForwardIterator I, OutputIterator O, Instrumented(UnaryPredicate) P, Procedure Proc>
void test_copy_if_complexity(Proc copy_if, I begin, I end, O out, P pred) {
    auto const n = std::distance(begin, end);

    // reset the predicate call count
    count<P>(Operation(P)::call) = 0;

    // call `copy_if`
    copy_if(begin, end, out, pred);

    // check that the predicate has been called the expected number of times
    assert(count<P>(Operation(P)::call) == n);
}
```

Of course, concept functions are composable. If it makes sense in our context, we could write for example:

```
template<Optional(Instrumented(UnaryPredicate)) P>
...
```

Also inside a concept function, a simpler example of creating a new concept is by using conjunction. Consider the concept `Serialize`:

```
template<typename T>
concept Serialize = requires(T const& t, std::ostream& o) {
    { serialize(t, o) } -> std::ostream&;
};
```

We can define a new concept by "and-ing" two concepts:

```
// Adds the `Serialize` constraints.
concept Serializable(concept C) {
    return C && Serialize;
};

// Now we can write:
template<Instrumented(Serializable(Container)) C>
...
```

`Serializable` is expressed as a concept function because any concept could be serializable: `Serializable(Container)`, `Serializable(Arithmetic)`, etc.

In [section 10. c](#), we wrote `ComplementType` with the following signature:

```
Predicate ComplementType(Predicate P) requires Class(P) {
    ...
}
```

We can now rewrite it as:

```
Predicate ComplementType((Predicate && Class) P) {
    ...
}
```

Or we can acknowledge that any concept could be required to only accept a class and use a concept function instead:

```
concept AsClass(concept C) {
    return C && Class;
}

Predicate ComplementType(AsClass(Predicate) P) {
    ...
}
```

Of course, disjunction can also be used to create concepts. For example in the following, the regex replacement can be specified as a string pattern or as a function object:

```
// Assumes `String` and `FunctionObject` concepts.
template<Range Rng, Regex Rgx, (String || FunctionObject) Rpl>
Predicate regex_replace(Rng rng, Rgx rgx, Rpl replace) {
```

This leads to the topic of composition. We now compose concept functions. We assume lambdas can be extended to do this job. The exact nature of `ConceptFunction` is to be investigated. For now, let's assume it just works:

```
// Compose two concept functions.
ConceptFunction Compose(ConceptFunction G, ConceptFunction F) {
    return [=](concept X) {
        return G(F(X));
    };
}
```

That could be used in the expected way to produce new concept functions:

```
concept InstrumentedSerializableClass = Compose(Instrumented, Compose(Serializable, AsClass));

template<Procedure Proc, InstrumentedSerializableClass(Container) C>
void test_my_algo(Proc p, C c) {
    // ...
}
```

We have the same potential readability problem as with type functions, so we can introduce an operator:

```
ConceptFunction operator|(ConceptFunction F, ConceptFunction G) {
    return Compose(G, F);
}

// And we can now alternatively write:
concept InstrumentedSerializableClass = AsClass | Serializable | Instrumented;
```

IV. Further considerations

1. Opportunity for unification

In the course of this paper, we strove to express similar ideas with similar syntaxes. For example, regardless of the nature of manipulated elements (objects, types...) we expressed:

- equality with ==
- assignment with =
- association between elements with functions, lambdas and operators
- conditional with if
- case matching with switch case
- loop with for

This makes code easy to read and understand. This makes the language simpler, easier to learn and more powerful. Any kind of entities introduced in the language (types, concepts...) should be checked against these manipulation patterns and these manipulation patterns should be expressed with syntaxes as close as possible.

Once syntax has been unified, new doors for unification open. For example, notice the similarity between object function composition, type function composition and concept function composition:

```
// Object function composition
template<FunctionObject G, FunctionObject F>
auto compose(G g, F f) {
    return [=](auto x) {
        return g(f(x));
    };
}

// Type function composition
TypeFunction Compose(TypeFunction G, TypeFunction F) {
    return [=](typename X) {
        return G(F(X));
    };
}

// Concept function composition.
ConceptFunction Compose(ConceptFunction G, ConceptFunction F) {
    return [=](concept X) {
        return G(F(X));
    };
}
```

If we had a mechanism to specify that a function can accept anything (objects, types, concepts), we could write a single version. Let's assume that @ denotes anything (object, type, concept), this would result in:

```

@ compose(@ g, @ f) {
    return [=](@ x) {
        return g(f(x));
    };
}

auto real = [](auto const& complex) {return complex.real();};

// Composition of object functions.
auto abs_of_real = compose(abs, real);

// Composition of type functions.
typename ValueTypeOfCodomainType = compose(ValueType, CodomainType);

// Composition of concept functions.
concept OptionalSerializable = compose(Optional, Serializable);

```

2. On metaprogramming

In this paper, we tried to unify manipulation of objects, manipulation of types and manipulation of concepts. Each of these happens at a different compilation stage. A schema of these stages could be:

```

token manipulation stage (preprocessor) -> concept manipulation stage ->
type manipulation stage -> object manipulation stage (runtime)

```

This is a pipeline and we could imagine adding more customization stages (a bit in the same way as the rendering pipeline on GPU got progressively more stages).

By adding the appropriate manipulation patterns (equality, functions, conditional...) for entities of each stage (preprocessor tokens, concepts, types, objects...), we do not negate the nature of each kind of entities: a concept is a concept, it is different from a type, which is different from a value. We only emphasize the similarity of manipulations.

This is the path of generic programming (in the STL sense): a generic template function does not try to convert its inputs to a common representation (by type-erasing them for example) but simply leverages the similarity of families of types in terms of syntax, semantics and space and time complexity.

A contrario with "meta types", we lower any entity to the object level. A type becomes a `meta::type` object, a concept could become a `meta::concept` object, etc. In a sense we are negating entities' nature, squashing any of them into objects, then manipulating them in this new form, and finally converting them back to their original form.

We believe the generic approach of this document is more natural. Still, the question of data structures arise. What should be the data structures for types? for concepts? We saw that type-for-loops rely on a kind of forward list to iterate. Is this data structure sufficient? In all probability, other data structures should be provided, in the same way some are adapted to the `constexpr` context.

Nevertheless, these considerations should not hide the fact that as a first step, being able to manipulate types in much the same way as objects would be a great language improvement. To attain more quickly this goal, one lead could be to consider the type function notation as a front-end to a "meta-type" implementation (see [implementation section](#)). This way, we would have both ease of manipulation and a reasonable implementation complexity.

V. Acknowledgements

Thanks to J. Lamotte for his review.

VI. References

[Elements of Programming] Alexander Stepanov & Paul McJones, 2009, Addison-Wesley

<http://www.elementsofprogramming.com>

[Boost.Hana] Louis Dionne, *Hana*

<https://github.com/boostorg/hana>

[P0425] Louis Dionne, *Metaprogramming by design, not by accident*

<http://open-std.org/JTC1/SC22/WG21/docs/papers/2017/p0425r0.pdf>

[P0707] Herb Sutter, *Metaclasses: Generative C++*

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0707r3.pdf>

[P0343] Vicente J. Botet Escribá, *Meta-programming High-Order functions*

<http://open-std.org/JTC1/SC22/WG21/docs/papers/2017/p0343r1.pdf>

[cppreference.com] Constraints and concepts

<http://en.cppreference.com/mwiki/index.php?title=cpp/language/constraints>

[P0694] Bjarne Stroustrup, *Function declarations using concepts*

<http://open-std.org/JTC1/SC22/WG21/docs/papers/2017/p0694r0.pdf>

[P0095] David Sankel, *Pattern Matching and Language Variants*

<http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0095r1.html>